

Лабораторная работа #5 – GDD, SRS, SDD/TDD

Часть 1: Создание Game Design Document (GDD) для простой компьютерной игры

Цель работы: Изучить процесс разработки Game Design Document (GDD) на примере простой 2D игры на Python с использованием библиотеки Pygame. Ознакомиться со структурой и содержанием GDD, научиться описывать игровые механики, сюжет, дизайн уровней и другие аспекты игрового дизайна.

Описание работы: Напишите GDD для простой 2D игры, разработанной с использованием Pygame. Документ будет включать подробное описание концепции игры, игровых механик, персонажей, уровней, пользовательского интерфейса и других важных элементов игрового дизайна.

Структура GDD

Game Design Document (GDD) служит руководством для всей команды разработки и описывает все аспекты игры. Ниже приведены основные разделы GDD с описанием и примерами.

1. Введение

- **Концепция игры:** Краткое описание идеи игры и ее уникальных особенностей.
- **Жанр и аудитория:** Определение жанра игры и целевой аудитории.
- **Платформы:** Указание платформ, на которых будет доступна игра.

Пример:

- Концепция игры: “Pixel Runner” — это 2D платформер, в котором игрок управляет персонажем, преодолевающим препятствия в пиксельном мире.
- Жанр: Платформер, аркада.
- Целевая аудитория: Игроки от 10 лет и старше, любящие ретро-игры.
- Платформы: ПК с операционными системами Windows, macOS и Linux.

2. Игровая механика

- **Основные действия игрока:** Описание того, что может делать игрок.
- **Управление:** Подробное описание схемы управления.
- **Правила игры:** Основные правила и цели игры.
- **Система прогресса:** Как игрок продвигается в игре, уровни, очки и т.д.

Пример:

- Основные действия: Бег, прыжок, уклонение от препятствий.
- Управление: Клавиши стрелок для движения, пробел для прыжка.
- Правила: Игрок должен добраться до конца уровня, избегая препятствий и собирая бонусы.
- Система прогресса: Игрок проходит уровни с возрастающей сложностью, собирает очки за собранные бонусы.

3. Сюжет и персонажи

- **Сюжет:** Краткое изложение истории игры.
- **Главный персонаж:** Описание протагониста, его мотивов и способностей.
- **Противники:** Описание врагов или препятствий, с которыми сталкивается игрок.
- **Второстепенные персонажи:** Если есть, описание других персонажей.

Пример:

- **Сюжет:** Персонаж путешествует по пиксельному миру, чтобы вернуть украденный артефакт.
- **Главный персонаж:** Маленький пиксельный герой с возможностью прыгать и бегать.
- **Противники:** Статичные и движущиеся препятствия, такие как шипы, падающие объекты.
- **Второстепенные персонажи:** Дружественные NPC, дающие советы.

4. Дизайн уровней (если они есть)

- **Общее описание:** Количество уровней, их тематика и сложность.
- **Элементы уровней:** Описание платформ, препятствий, интерактивных объектов.
- **Прогрессия сложности:** Как меняется сложность от уровня к уровню.

Пример:

- **Общее описание:** Игра состоит из 10 уровней, каждый с уникальным дизайном.
- **Элементы уровней:** Подвижные платформы, скрытые проходы, собираемые бонусы.
- **Прогрессия сложности:** Увеличение скорости движения препятствий, добавление новых видов препятствий на более высоких уровнях.

5. Графика и звук

- **Стиль графики:** Описание визуального стиля игры (например, пиксель-арт).
- **Анимации:** Перечень необходимых анимаций для персонажей и объектов.
- **Звуковые эффекты:** Описание звуков для действий и событий.
- **Музыка:** Фоновая музыка, соответствующая атмосфере игры.

Пример:

- **Стиль графики:** Ретро пиксель-арт с яркими цветами.
- **Анимации:** Бег, прыжок, падение главного персонажа; анимации препятствий.
- **Звуковые эффекты:** Звук прыжка, сбор бонуса, столкновения с препятствием.
- **Музыка:** Энергичная 8-битная музыка на каждом уровне.

6. Пользовательский интерфейс (UI)

- **Элементы UI:** Описание экранов меню, HUD (Heads-Up Display), кнопок и других элементов интерфейса.
- **Навигация:** Как пользователь перемещается по меню и выбирает опции.
- **Отображение информации:** Как в игре отображаются очки, жизни, уровень и т.д.

Пример:

- **Элементы UI:** Главное меню с опциями “Начать игру”, “Настройки”, “Выход”; интерфейс во время игры с отображением очков и количества жизней.
- **Навигация:** Управление в меню с помощью клавиш стрелок и клавиши Enter для выбора.
- **Отображение информации:** Очки и жизни отображаются в верхнем левом углу экрана.

7. Технические требования

- **Платформа и требования к системе:** Подробности о поддерживаемых ОС и минимальных системных требованиях.
- **Используемые технологии:** Перечисление языков программирования, библиотек и инструментов разработки.
- **Ограничения:** Любые технические ограничения, влияющие на разработку.

Пример:

- **Платформа:** ПК с Windows 7+, macOS 10.12+, Linux с Python 3.x.
- **Минимальные системные требования:** CPU 1.5 ГГц, 2 ГБ ОЗУ, 100 МБ свободного места на диске.
- **Используемые технологии:** Python 3.8, Pygame 2.0, редактор кода Visual Studio Code.

- Ограничения: Игра должна работать в разрешении 800x600 пикселей.

8. Планирование и этапы разработки

- **Этапы разработки:** Разбивка проекта на фазы с описанием задач.
- **Сроки:** Оценка времени, необходимого для каждого этапа.
- **Ресурсы:** Какие ресурсы и навыки необходимы для реализации проекта.

Пример:

- Этапы разработки:
 - Концептуализация и разработка GDD (1 неделя)
 - Прототипирование основных механик (2 недели)
 - Разработка уровней и контента (3 недели)
 - Тестирование и отладка (1 неделя)
- Сроки: Общая продолжительность проекта — 7 недель.
- Ресурсы: 1 программист, 1 художник, доступ к компьютеру с установленным ПО.

9. Тестирование и обеспечение качества

- **План тестирования:** Как будет проводиться тестирование игры.
- **Критерии успешности:** Что считается успешным завершением разработки.
- **Обратная связь:** Как будет собираться и учитываться обратная связь от тестировщиков.

Пример:

- План тестирования: Проведение юнит-тестов для основных механик, бета-тестирование с участием группы пользователей.
- Критерии успешности: Игра работает без критических ошибок, соответствует описанным в GDD механикам.
- Обратная связь: Сбор отзывов через опросы, исправление выявленных проблем.

10. Приложения

- **Эскизы и диаграммы:** Визуальные материалы, помогающие понять дизайн игры.
- **Списки ресурсов:** Перечень необходимых графических и звуковых ресурсов.
- **Ссылки на примеры:** Ресурсы и игры, вдохновившие проект.

Пример:

- Эскизы персонажей и уровней.
- Список требуемых спрайтов и звуковых эффектов.
- Ссылки на классические платформы для вдохновения.

Заключение

Game Design Document (GDD) является ключевым документом, определяющим все аспекты будущей игры. Его тщательная разработка обеспечивает ясное понимание проекта всеми участниками команды и служит основой для успешной реализации игры.

Рекомендуемые источники для примеров GDD:

1. [Game Design Document - Project “Vanishing”](#)
2. [Pid Game Design Document](#)
3. [Game Design Document Template by Unity](#)
4. [Zombie Island - Game Design Document](#)
5. [Game Concept & Design Document Template](#)

Примечание: При написании GDD используйте четкие и понятные формулировки. Документ должен быть доступен для понимания всеми членами команды разработки и вами прежде всего.

Часть 2: Создание документа SRS для игры (продукта)

Цель работы:

Ознакомиться с процессом разработки спецификации требований к программному обеспечению (SRS) на примере простой 2D игры на Python с использованием библиотеки Pygame. Освоить структуру и содержание документа SRS, научиться формулировать требования к программному продукту.

Описание работы:

В ходе этой части лабораторной работы нужно создать документ SRS для простой 2D игры, разработанной с использованием Pygame. Документ будет включать основные разделы спецификации требований, такие как введение, описание системы, функциональные и нефункциональные требования.

Частично в описательную часть требования к продукту вы перенесёте сюда содержимое из своего GDD (Game Design Document).

Структура SRS

Документ SRS должен следовать стандартной структуре, включающей несколько основных разделов. Ниже приведены эти разделы с кратким описанием и примерами.

1. Введение

Раздел “Введение” служит для общего описания игры и контекста разработки. Здесь задаются основные параметры, такие как цели проекта, целевая аудитория, функциональные ограничения и другие важные аспекты.

Пример: - Цель игры — создать простую аркадную игру, где игрок управляет персонажем, который должен избегать препятствий. - Целевая аудитория: пользователи, интересующиеся аркадными играми на ПК. - Ограничения: игра должна быть разработана на Python с использованием Pygame, поддерживать разрешение экрана 800x600 пикселей.

2. Общее описание

Этот раздел описывает общую функциональность системы, её задачи и ключевые характеристики. Включает описание связей с другими системами (например, интеграция с базой данных или игровыми платформами).

Пример: - Игра будет состоять из одного игрового экрана, на котором движется главный персонаж и появляются случайные препятствия. - Игровой процесс будет основываться на наборе очков за пройденное время и успешные уклонения от препятствий.

3. Функциональные требования

Здесь перечисляются конкретные функции игры, которые должны быть реализованы. Каждый пункт функциональных требований должен быть конкретным и измеримым.

Пример: - Игрок должен управлять движением персонажа с помощью клавиш-стрелок. - Препятствия появляются каждые 3 секунды и движутся по направлению к игроку. - При столкновении с препятствием игра заканчивается.

4. Нефункциональные требования

Включает характеристики игры, не относящиеся напрямую к её функциональности, такие как производительность, совместимость, безопасность и поддерживаемые платформы.

Пример: - Игра должна поддерживать плавную анимацию с частотой кадров не менее 30 FPS. - Приложение должно работать на операционных системах Windows и Linux.

5. Ограничения

Описываются внешние или внутренние ограничения, которые могут влиять на процесс разработки игры. Например, ограничения по времени, ресурсам или технологиям.

Пример: - Игра должна быть завершена в течение 4 недель. - Размер игрового файла не должен превышать 50 МБ.

6. Требования к интерфейсу пользователя

Этот раздел описывает, как игра будет взаимодействовать с пользователем, включая элементы управления, интерфейсные элементы и их расположение.

Пример: - Основной игровой экран будет содержать кнопку “Старт” для начала игры и кнопку “Выход” для завершения работы программы. - Счёт будет отображаться в верхнем правом углу экрана.

7. Диаграммы и модели

Диаграммы помогают визуализировать архитектуру и логику системы. Можно использовать диаграммы состояний для описания игровых состояний (начало игры, игра, конец игры) или диаграммы классов для моделирования архитектуры кода.

Пример: - Диаграмма состояний: начальное состояние — главное меню, затем переходит в состояние игры, а после завершения игры — в экран с результатами.

8. Требования к производительности

Здесь указываются характеристики производительности, такие как время отклика игры, требования к памяти и процессору.

Пример: - Время загрузки игры не должно превышать 2 секунд. - Игра должна потреблять не более 100 МБ оперативной памяти во время выполнения.

9. Правила и ограничения

Правила и ограничения могут включать юридические аспекты, лицензирование, требования безопасности и условия распространения.

Пример: - Игра должна быть выпущена под лицензией MIT. - Все внешние ресурсы (графика, музыка) должны быть использованы в соответствии с лицензией CC BY-SA 4.0.

10. Сценарии использования (Use Case)

Сценарии использования описывают взаимодействие пользователя с игрой в конкретных ситуациях. Это помогает лучше понять, как игрок будет использовать игру на разных этапах.

Пример: - Сценарий: игрок запускает игру, выбирает уровень сложности, играет и получает итоговый счёт после завершения уровня.

Итого

Каждый раздел SRS документа должен быть написан строго и чётко. Важно, чтобы все требования были понятны и проверяемы. Это обеспечит успешную реализацию игры в рамках проекта. Использование стандартных шаблонов поможет поддерживать единообразие и полноту спецификации.

Рекомендуемые источники для примеров SRS:

1. [IEEE Guide to SRS](#)
2. [IEEE Guide to SRS - 2018](#)
3. [IEEE Template on GitHub](#)
4. [Ещё один шаблон на GitHub](#)
5. [Поищите примеры шаблонов самостоятельно](#)
6. [SRS: что такое и зачем это нужно разработчикам - VC.RU](#)
7. [Старая статья на Habr - “Правила составления Software requirements specification”](#)
8. [SRS: что такое и зачем это нужно разработчикам - YuSMPgroup.ru](#)
9. [Как написать спецификацию требований к программному обеспечению - stfalcon.com](#)
10. [Пример SRS для набора мини-игр Astropolis](#)

Часть 3: Создание Software Design Document (SDD)

Цель работы: Изучить процесс разработки Software Design Document (SDD) на примере простой 2D игры на Python с использованием библиотеки Pygame. Ознакомиться со структурой и содержанием SDD, научиться описывать архитектуру и проектные решения программного продукта.

Описание работы: Создайте SDD для простой 2D игры, разработанной с упором на Pygame (если знаете другие игровые движки - то можно использовать их). Документ будет включать подробное описание архитектуры системы, проектных решений и деталей реализации основных компонентов.

Структура SDD

Документ SDD должен содержать детальное описание архитектуры программного продукта, его модулей и компонентов. Ниже приведены основные разделы SDD с описанием и примерами.

Существуют и другие (более высокоуровневые) типы документов, такие как Software Architecture Document, но в нашем случае мы ограничиваемся только SDD, в который можно включить в том числе элементы архитектурного описания вашего приложения.

1. Введение

- **Цель документа:** Описать архитектуру и проектные решения для разработки 2D игры.
- **Область применения:** Определить границы системы, описать, что входит в проект.
- **Ссылки:** Перечислить документы и ресурсы, использованные при разработке (например, SRS документация, стандарты).

Пример:

- Цель документа: Предоставить подробное описание проектных решений для 2D игры “Space Adventure” на Python с использованием Pygame.
- Область применения: Документ предназначен для разработчиков, участвующих в создании игры.
- Ссылки: SRS документация для игры “Space Adventure”, или ссылка на ваш документ SRS из первой части лабораторной работы.

2. Общее описание системы

- **Обзор системы:** Кратко описать систему, ее функции и возможности.
- **Архитектурные ограничения:** Указать ограничения, влияющие на дизайн (например, использование определенных технологий).

Пример:

- Игра представляет собой космический шутер, где игрок управляет космическим кораблем и уничтожает астероиды.
- Игра разрабатывается на Python 3.11+ с использованием Pygame 2.6+.

3. Архитектурный дизайн

- **Архитектурный подход:** Описать выбранный архитектурный стиль (например, объектно-ориентированная архитектура).
- **Диаграмма архитектуры:** Предоставить диаграмму, показывающую основные компоненты и их взаимодействие. Можно рисовать диаграммы компонентов, классов, диаграммы последовательности и так далее, а также диаграмму развертывания, если необходимо.

Пример:

- Архитектура системы основана на объектно-ориентированном подходе с использованием основных классов для игровых объектов.
- Диаграмма архитектуры отображает классы: Game, Player, Asteroid, Bullet, и их взаимодействие.

4. Детальное описание компонентов

Здесь приведено только примерное краткое описание гипотетических компонентов и объектов/классов игры. Только от вас зависит её настоящее наполнение в плане объектов, их логика/поведение и взаимодействие с игроком.

4.1. Класс Game

- **Описание:** Управляет основным игровым циклом и состояниями игры.
- **Методы:**
 - `run()`: Запускает игровой цикл.
 - `handle_events()`: Обработывает события ввода.
 - `update()`: Обновляет состояние игры.
 - `draw()`: Отрисовывает текущий кадр.

Пример:

- Метод `run()` содержит цикл `while`, который вызывает `handle_events()`, `update()`, и `draw()` в каждой итерации.

4.2. Класс Player

- **Описание:** Представляет космический корабль игрока.
- **Атрибуты:** Позиция (x, y), скорость, изображение спрайта.
- **Методы:**
 - `move_left()`: Перемещает корабль влево.
 - `move_right()`: Перемещает корабль вправо.
 - `shoot()`: Создает объект `Bullet`.

Пример:

- При нажатии клавиши `SPACE` вызывается метод `shoot()`, который создает новый объект `Bullet` и добавляет его в список пуль.

4.3. Класс Asteroid

- **Описание:** Представляет астероиды в игре.
- **Атрибуты:** Позиция, скорость, размер, изображение спрайта.
- **Методы:**
 - `update()`: Обновляет позицию астероида.
 - `draw(screen)`: Отрисовывает астероид на экране.

Пример:

- Астероиды появляются в верхней части экрана с случайной позицией по оси x и движутся вниз с постоянной скоростью.

4.4. Класс Bullet

- **Описание:** Представляет пули, выпущенные игроком.
- **Атрибуты:** Позиция, скорость, изображение спрайта.
- **Методы:**
 - `update()`: Обновляет позицию пули.
 - `draw(screen)`: Отрисовывает пулю на экране.

Пример:

- Пуля движется вверх (как в Space Invaders) с постоянной скоростью и уничтожается при выходе за пределы экрана.

5. Дизайн данных

- **Структуры данных:** Описать используемые структуры данных для хранения игровых объектов.
- **Формат данных:** Указать форматы хранения настроек или ресурсов (если применимо).

Пример:

- Списки `asteroids` и `bullets` используются для хранения активных астероидов и пуль соответственно.
- Позиции объектов хранятся в виде координатных пар (x, y) .

6. Интерфейсы системы

- **Внутренние интерфейсы:** Описать взаимодействие между классами и модулями.
- **Внешние интерфейсы:** Описать взаимодействие с пользователем.

Пример:

- Класс `Game` взаимодействует с классами `Player`, `Asteroid`, `Bullet` для обновления состояния игры.
- Пользователь взаимодействует с игрой через клавиатуру (клавиши стрелок для движения, `SPACE` для стрельбы).

7. Пользовательский интерфейс

- **Описание интерфейса:** Описать внешний вид игры и элементы управления.
- **Макеты экранов:** Предоставить описания основных экранов игры (например, главный экран, экран паузы).

Пример:

- Игровой экран отображает космический корабль внизу, астероиды, падающие сверху, и текущий счет в верхнем левом углу.
- Управление кораблем осуществляется с помощью клавиш `LEFT` и `RIGHT`.

8. Требования к производительности

- **Производительность системы:** Указать ожидаемую частоту кадров и время отклика.
- **Ограничения ресурсов:** Указать максимальное использование памяти и процессора.

Пример:

- Игра должна поддерживать стабильную частоту кадров не менее 60 FPS.
- Потребление оперативной памяти не должно превышать 150 МБ.

9. Безопасность

- **Обработка ошибок:** Описать, как система обрабатывает возможные ошибки и исключения.
- **Валидация данных:** Указать меры по предотвращению некорректных данных.

Пример:

- Используются блоки try-except для обработки ошибок загрузки ресурсов.
- Проверяется наличие необходимых файлов ресурсов при запуске игры.

10. Обеспечение качества

- **Стратегия тестирования:** Описать план тестирования компонентов игры.
- **Критерии приемки:** Указать условия, которые должны быть выполнены для успешной реализации.

Пример:

- Разработаны юнит-тесты для классов Player, Asteroid, Bullet.
- Игра проходит стресс-тестирование с большим количеством астероидов без снижения производительности.

11. Требования к среде разработки

- **Инструменты разработки:** Перечислить используемые IDE, фреймворки, библиотеки и инструменты.
- **Ограничения среды:** Указать версии языков и библиотек.

Пример:

- Язык программирования: Python 3.11+.
- Библиотеки: Pygame 2.6+.
- Среда разработки: PyCharm Community Edition.

12. Приложения

- **Диаграммы UML:** Включить диаграммы классов и последовательностей. Сюда можно вынести наиболее громоздкие диаграммы, сославшись на них из основного текста SDD.
- **Дополнительные материалы:** Включить схемы или таблицы, помогающие понять дизайн.

Пример:

- Диаграмма классов отображает отношения между Game, Player, Asteroid, Bullet.
- Диаграмма последовательности описывает взаимодействие при обновлении игрового состояния в каждом цикле.

Итоги

SDD должен предоставить подробное описание архитектуры и проектных решений игры. Четкое и детальное изложение облегчит разработку и дальнейшую поддержку программного продукта.

Различные примеры шаблонов SDD, примеров и статей

1. [SDD Virtual Simulation Example](#)
2. Примеры на GitHub:
 - [jam01](#)
 - [jrocha-dev](#)
 - [AdriaticOrg - укороченный шаблон для создания многих SDD сразу - для фич](#)
 - Можете также самостоятельно поискать многочисленные примеры.
3. [Software Design Documentation Template - bit.ai](#)

4. [How to Create a Software Design Document? - https://clickup.com/](https://clickup.com/)

Также присмотритесь к документам TDD. Пример одного из них: [Technical Design Document Example](#)

Примечание: При написании SDD используйте понятные формулировки и стандартизированные термины. Это обеспечит ясность и доступность документа для всех участников разработки.